

1.4 Word Embeddings and Semantic Geometry

Dense vectors from a corpus window, skip-gram softmax, analogies, and a signed-projection bias probe.

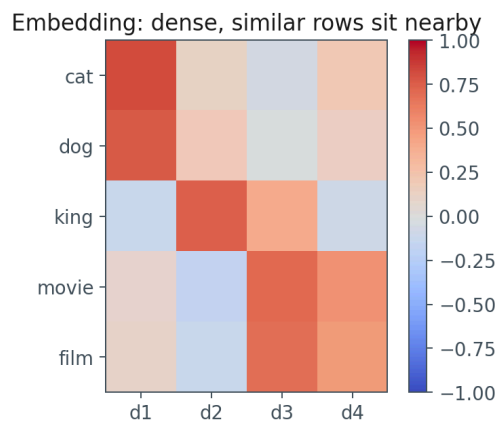
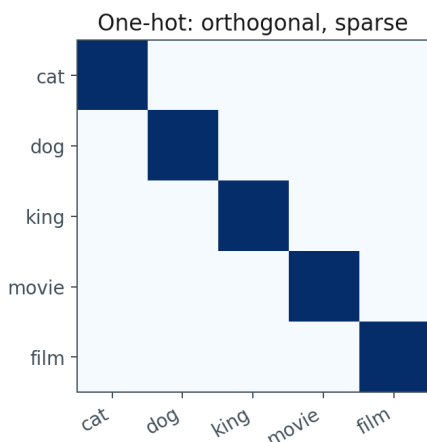
These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

A language model does not read characters as characters. It reads **vectors**. This week those vectors are word embeddings. From Week 3 they are token embeddings inside a transformer. The geometry is the same idea. By the end you should be able to write skip-gram's softmax, compute a cosine, and run Lab 1's occupation probe.

1. What an embedding is

An **embedding** is a map from a token id to a dense vector. Give each word its own one-hot coordinate and *film* and *movie* are orthogonal: inner product 0. The vector is sparse, as long as the vocabulary, and a new word has no coordinate at all.

A **distributed** representation is short and dense (50–300 dimensions is a classical size; transformer hidden sizes are larger). Meaning is spread across coordinates. Similar usage produces nearby vectors. That is the geometry you can learn.



Distributional is about the data: words that occur in similar contexts mean similar things (Harris).

Distributed is about the code: a dense vector. An **embedding** is the map from a token id into that vector. Write those three words on the board as a triangle. Mixing them is the most common exam slip this week.

If the map is good, **direction means something**. The Mikolov analogy is the slogan:

$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}.$$

The same offset can look like gender, tense, or capital-of. Week 3 will replace static rows with contextual states; the inner-product geometry does not go away.

2. Why we use it

One-hot geometry cannot say that *film* and *movie* are similar. You need a space where cosine is a useful neighbor query, analogies are a debug tool, and a downstream classifier can use a short vector instead of a V -dimensional sparse indicator.

You do not type a definition. You **count or predict company**. Firth: you shall know a word by the company it keeps. Counting co-occurrences and then factoring the matrix is one distributional story. This course uses the predictive story (skip-gram) because it is the same training loop as an LLM.

Nearest neighbors are how you debug. If *Paris* sits next to *France* and *Rome*, training learned something. If it sits next to punctuation, the corpus or the window is wrong.

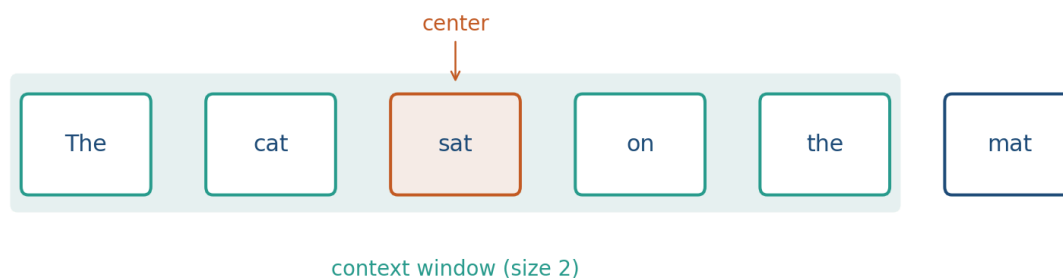
3. Architecture

The embedding is a table $E \in \mathbb{R}^{V \times d}$. Row i is the vector for token i . Input is a token id.

`nn.Embedding(V, d)` is a lookup: row c of that matrix. A one-hot times the matrix is the same map.

Word2Vec trains a small two-layer net on a fake task. **Skip-gram**: from the center word, predict each neighbor in a window. **CBOW**: from the neighbors, predict the center. After training, **keep the center rows**. Those rows are the embeddings. Throw away the softmax classifier (U).

Skip-gram: predict each neighbor from the center word



Let v_c be the **input** (center) embedding of the center word, and u_o the **output** embedding of a neighbor. You do not build a huge co-occurrence matrix and then factor it (that is LSA / GloVe; we do not assign SVD this week). A full softmax over 100,000 words is slow. Negative sampling (Mikolov 2013b) replaces it. We cite that paper; we do not derive it this week.

A transformer does not throw the rest of the net away: it *keeps* transforming those vectors. Word2Vec is the geometry without the rest of the stack.

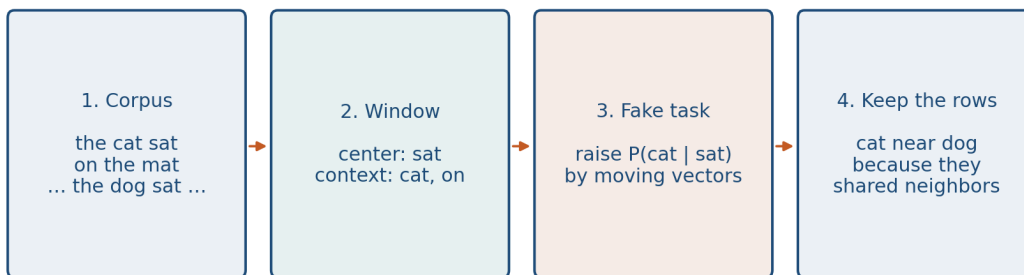
4. How it works, step by step

How a distributional embedding is obtained from a corpus:

1. Take a corpus of running text.
2. At each position, treat one word as the **center** and the nearby words as **context** (a window).
3. Train a fake task: from the center, predict a neighbor (skip-gram), or the reverse (CBOW). Words that appear in similar windows get similar vectors.
4. After training, **keep the center rows**. Throw away the softmax classifier.

That is why *cat* sits near *dog*: they kept similar neighbors, not because someone labeled them as animals.

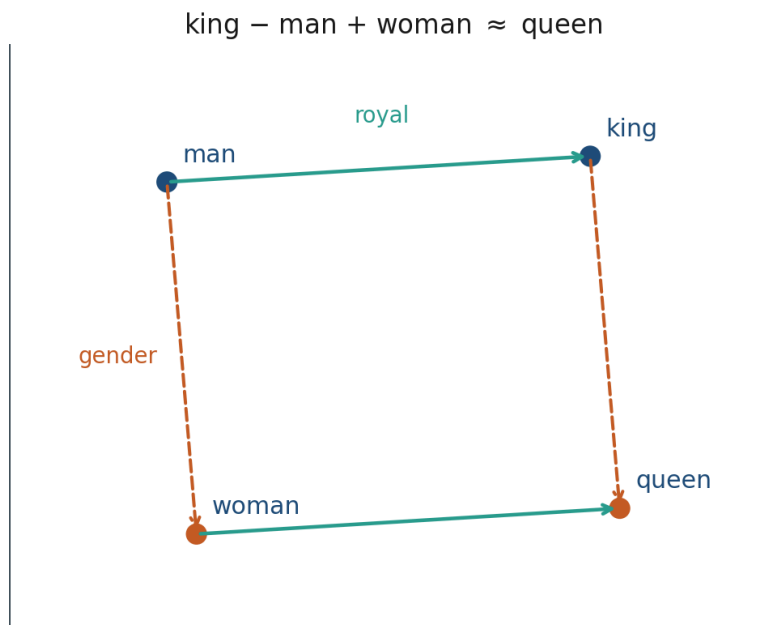
Distributional embeddings are obtained from running text, not from a thesaurus



You measure similarity with **cosine**, not Euclidean distance (length is partly frequency):

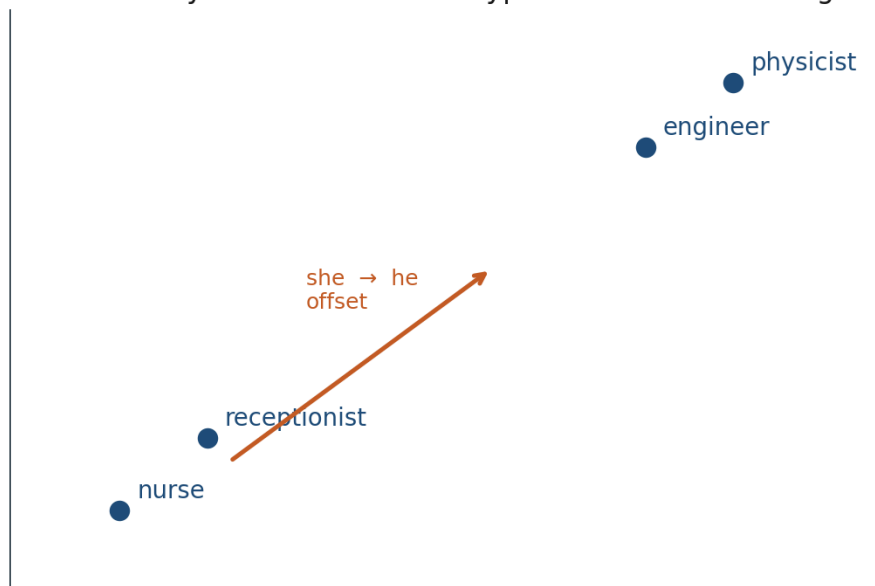
$$\cos(u, v) = \frac{u^\top v}{\|u\| \|v\|}.$$

It ignores length. That is why a frequent word with a long vector can still sit next to a rare synonym.



The same offsets that encode “capital of” can encode stereotypes. Occupation words often sit along a she/he direction in older embeddings. That is not a bug in cosine; it is a property of the training text.

Geometry can encode stereotypes as well as meaning



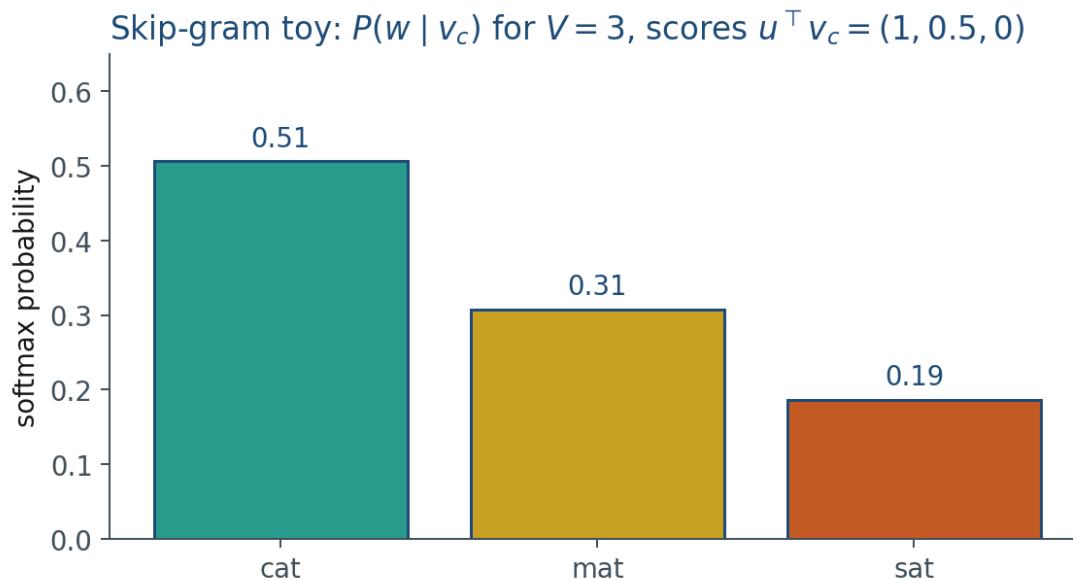
Lab 1’s probe is the signed projection onto $o = \vec{he} - \vec{she}$:

$$\text{score}(v) = \frac{v^T o}{\|o\|}.$$

A larger score sits closer to *he* in this constructed space. Debiasing is incomplete; reporting the probe is part of the project writeup when your system ranks people, jobs, or medical outcomes.

Tiny vocabulary {cat, mat, sat}. Take center *the* with $v_c = (1, 0)$, and output vectors $u_{\text{cat}} = (1, 0)$, $u_{\text{mat}} = (0.5, 0)$, $u_{\text{sat}} = (0, 1)$. The three scores are 1, 0.5, 0. Softmax:

$$p \propto (e^1, e^{0.5}, e^0) \approx (2.718, 1.649, 1), \quad p \approx (0.51, 0.31, 0.19).$$



5. Mathematical formulas

Skip-gram's softmax (Mikolov §§1–3) is the same softmax as note 1.2, with logits $z_w = u_w^\top v_c$:

$$P(w_o \mid w_c) = \frac{\exp(u_o^\top v_c)}{\sum_w \exp(u_w^\top v_c)}.$$

Cosine similarity:

$$\cos(u, v) = \frac{u^\top v}{\|u\| \|v\|}.$$

Bias probe used in Lab 1, with $o = \overrightarrow{\text{he}} - \overrightarrow{\text{she}}$:

$$\text{score}(v) = \frac{v^\top o}{\|o\|}.$$

The analogy arithmetic is vector addition in the same space: $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$.

6. Positive points and negative points

Positive.

- Analogies and nearest neighbors are a cheap debug tool: if *Paris* sits next to *France*, training learned something.
- Cosine ignores length, so frequency-driven vector magnitude does not dominate a neighbor query.
- Skip-gram uses the same softmax-and-loss loop as an LLM, so this week is not a dead-end method.

- A $V \times d$ lookup is the token embedding table inside a transformer as well.

Negative.

- Static vectors: one vector per word type, so *bank* (river) and *bank* (money) share a row. Week 3 replaces this with contextual states.
- Bias in the corpus becomes geometry. Occupation probes along $\vec{he} - \vec{she}$ are a property of the text, not a cosine bug.
- Euclidean distance makes frequent (long) vectors look far from rare synonyms.
- Analogy accuracy is an intrinsic score; it need not predict your project's downstream metric.
- A full softmax over a large V is slow (negative sampling exists; we do not derive it this week).

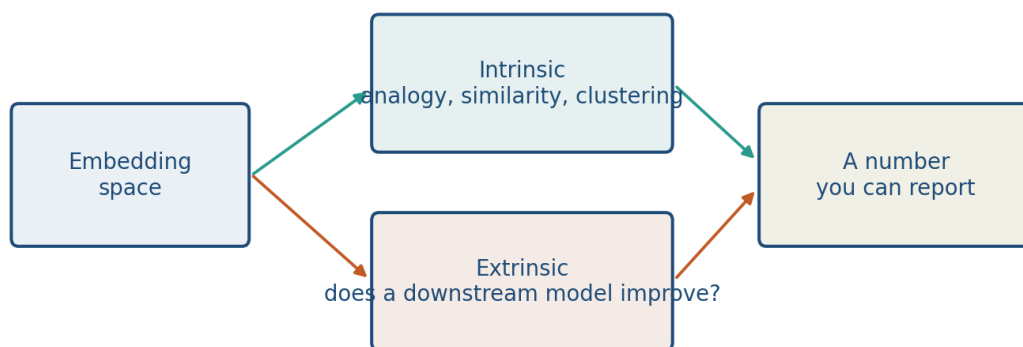
When not to. If you need a vector that depends on the sentence, a static table is the wrong object. That is why Week 3's contextual states exist.

7. Intrinsic versus extrinsic evaluation

Intrinsic: does the space look right on its own? Analogies, word similarity datasets, clustering of synsets.

Extrinsic: plug the vectors into a downstream model (sentiment, retrieval, NER). If the task score moves, the embedding helped that task.

Two ways to score embeddings. They need not agree.



They need not agree. A space that wins analogies can still be a weak feature for your classifier. For a project, extrinsic is the number that matters. Intrinsic is how you debug.

8. Teaching this note

~18 minutes. One-hot vs dense, then the four-box pipeline (corpus → window → fake task → keep the rows), then skip-gram's $P(o | c)$ on the $V = 3$ softmax, then cosine, then the occupation projection so

Lab 1's probe is not a surprise. StatQuest Word2Vec is **homework (0:00–12:00)**, skip-gram vs. CBOW).

9. Worked example

Three 2-D vectors:

$$\overrightarrow{\text{movie}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \overrightarrow{\text{film}} = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \quad \overrightarrow{\text{pizza}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

$\text{Cosine}(\text{movie}, \text{film}) = 0.8 / \sqrt{0.68} \approx 0.97$. $\text{Cosine}(\text{movie}, \text{pizza}) = 0$. One-hot versions of the same three words would all be orthogonal.

Bias probe aligned with Lab 1: $o = \overrightarrow{\text{he}} - \overrightarrow{\text{she}}$. If $\overrightarrow{\text{he}} = (1, 0)$, $\overrightarrow{\text{she}} = (0, 1)$, then $o = (1, -1)$ and $\|o\| = \sqrt{2}$. For $\overrightarrow{\text{engineer}} = (0.9, 0.1)$ and $\overrightarrow{\text{nurse}} = (0.1, 0.8)$,

$$\text{score}(\text{engineer}) = \frac{0.9 - 0.1}{\sqrt{2}} \approx 0.57, \quad \text{score}(\text{nurse}) = \frac{0.1 - 0.8}{\sqrt{2}} \approx -0.49.$$

Engineer sits toward *he*; nurse sits toward *she*. The CSV in Lab 1 was built to make that split visible. On a real model you report the same probe, not this cartoon.

10. Where students get stuck

- Using Euclidean distance and then “the frequent word is never nearest.”
 - Calling the one-hot vector an embedding because it is a vector.
 - Treating analogy accuracy as the project metric.
 - Thinking an embedding file appeared without a corpus: skip the four-box pipeline.
-

11. Video

Watch [StatQuest: Word Embedding and Word2Vec, Clearly Explained](#).

Pause on skip-gram vs. CBOW, and on the moment the hidden weights become the embedding table. You can skip the neural-net-from-scratch recap if note 1.2 already landed.

12. Practice

1. In one sentence each: distributional, distributed, embedding.
2. Why is cosine the default similarity?
3. Give one intrinsic and one extrinsic test for a word embedding. Which one would you put in a project report?

4. Compute cosine between $a = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and $b = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$. Then compute $a^\top b$. Which number changed more when you replace b by $10b$?
5. In 2-D, $\overrightarrow{\text{king}} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$, $\overrightarrow{\text{man}} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$, $\overrightarrow{\text{woman}} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$. Compute $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}}$. What point would you hope $\overrightarrow{\text{queen}}$ near?
6. With $v_c = (1, 0)$ and scores $u^\top v_c = (1, 0.5, 0)$ for cat, mat, sat, which word does skip-gram call most likely? Write the three softmax probabilities to two decimals.
7. In four short lines: how is a distributional embedding obtained from a corpus? (Corpus, window, fake task, what you keep.)